

SSAFY

Git

Revert & Reset

이제



목 차

Revert & Reset

- Git revert
- Git reset

참고

- 바로 직전 생성한 commit 수정하기
- 파일 내용을 수정 전으로 되돌리기
- Staging area에 올라간 파일을 unstage 하기
- 삭제한 commit으로 돌아가기

Revert & Reset

Git revert

Git revert

Git revert

특정 commit을 없었던 일로 만드는 작업

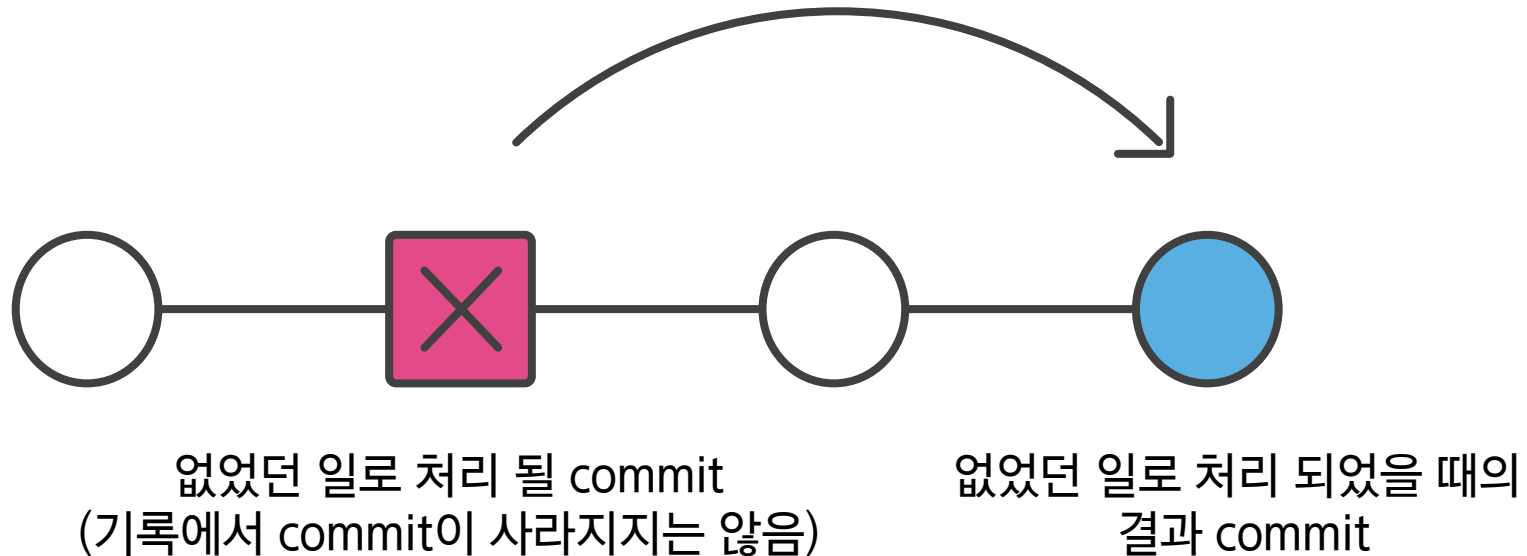
 Git revert

git revert <commit id>

<commit id>에 해당하는 commit을 없었던 일로 되돌리기

Git revert 작동 원리

- "재설정"
- 단일 commit을 실행 취소하는 것
- 프로젝트 기록에서 commit을 없었던 일로 처리 후 그 결과를 새로운 commit으로 추가함



revert 실습 (1/6)

- 제공된 revert-reset-practice.zip 중 revert 폴더에서 진행

revert 실습 (2/6)

- Commit 목록 확인

```
$ git log --oneline  
d7c8501 (HEAD -> master) third  
91cbd74 second  
f7b3a3d first
```

revert 실습 (3/6)

- Commit 별 내역

Commit 이름	Commit 내역
first	1.txt가 생성된 사실 기록
second	2.txt가 생성된 사실 기록
third	3.txt가 생성된 사실 기록

- untracked.txt
 - Commit에 포함된 적 없는 파일

revert 실습 (4/6)

- 'second' commit을 없었던 일로 만들기

```
$ git revert 91cb
```

TIP

Git commit의 commit id는 40글자로 이루어진 해시로 만들어집니다.

- Revert와 같이 특정 commit을 지정해야 하는 경우 이 commit id를 제공해야 하지만, 전체 id를 제공할 필요는 없습니다.
- 서로 다른 commit이 구분될 수 있도록 앞에서부터 일부분을 작성해야 하며, 최소 4자를 작성합니다.

revert 실습 (5/6)

- vim 편집기 실행
 - 'second' commit을 없었던 일로 만든 결과를 기록한 새로운 commit을 작성하기 위함

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

bash + ▾ □ 🗑️ ⋮ | [] ×

Revert "second"

This reverts commit 91cbd74ec969ce40577658673789755440b7bd18.

Please enter the commit message for your changes. Lines starting
with '#' will be ignored, and an empty message aborts the commit.

On branch master
Changes to be committed:
deleted: 2.txt

Untracked files:
untracked.txt

~ ~ ~ ~ ~ ~ ~ ~

.git/COMMIT_EDITMSG [unix] (20:07 06/01/2026) 14,1 All
~/Desktop/reset-revert-practice/revert/.git/COMMIT_EDITMSG" [unix] 14L, 322B

revert 실습 (6/6)

- vim 편집기 저장 후 종료
- 'second' commit을 없었던 일로 만든 결과 확인
 - 2.txt 사라짐
 - 'second' commit은 삭제되지 않고 새로운 commit이 작성됨

```
$ git log --oneline
d7c8501 (HEAD -> master) Revert "second"
d7c8501 third
91cbd74 second
f7b3a3d first
```

REVERT

1.txt

3.txt

untracked.txt

U

Git Revert 정리

- 변경 사항을 안전하게 실행 취소할 수 있도록 도와주는 순방향 실행 취소 작업
- Commit 기록에서 commit을 삭제하거나 분리하는 대신,
지정된 변경 사항을 반전시키는 새 commit을 생성
- Git에서 기록이 손실되는 것을 방지하며 기록의 무결성과 협업의 신뢰성을 높임

Revert 추가 활용법

```
# 공백을 사용해 여러 commit을 한꺼번에 실행 취소 가능
$ git revert 7f6c24c 006dc87 3551584

# `..`을 사용해 범위를 지정하여 여러 commit을 한꺼번에 실행 취소 가능
$ git revert 3551584..7f6c24c

# commit 메시지 작성을 위한 편집기를 열지 않음 (자동으로 commit 진행)
$ git revert --no-edit 7f6c24c

# 자동으로 commit 하지 않고, Staging Area에만 올림 (이후에 직접 commit 해야 함)
# 이 옵션은 여러 commit을 revert 할 때 하나의 commit으로 묶는 것이 가능
$ git revert --no-commit 7f6c24c
```

Git reset

Git reset

Git reset

특정 commit으로 되돌아가는 작업

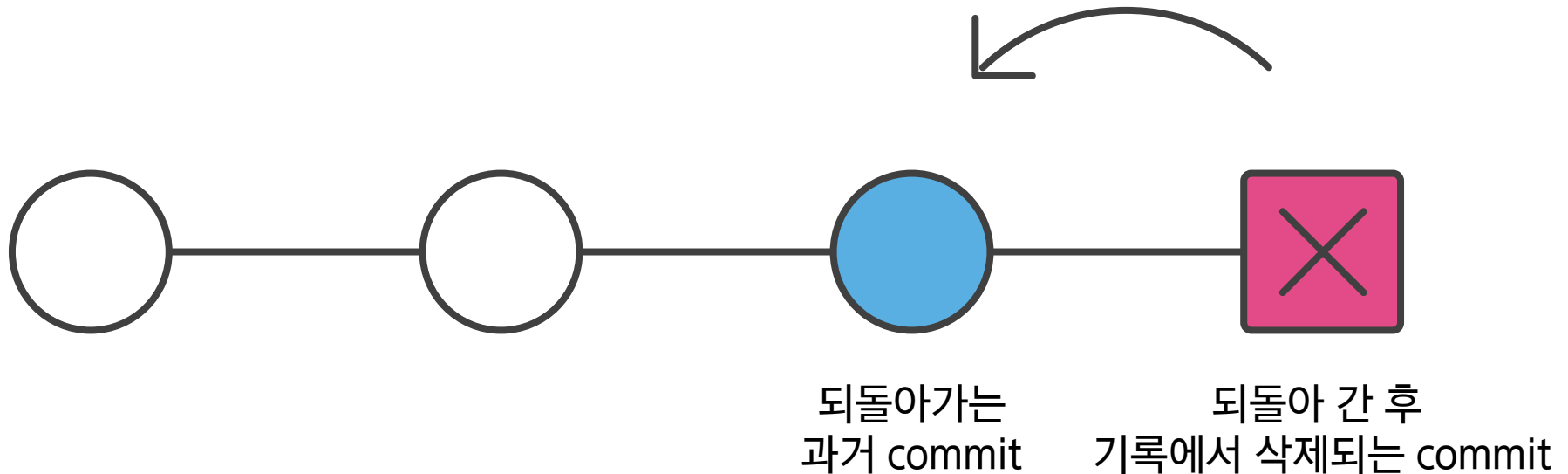
 Git reset

git reset [옵션] <commit id>

<commit id>까지 되돌아가기

Git reset 작동 원리

- "되돌리기"
- 시계를 마치 과거로 돌리는 듯한 행위
- 특정 commit으로 되돌아 갔을 때, 되돌아간 commit 이후의 commit은 모두 삭제



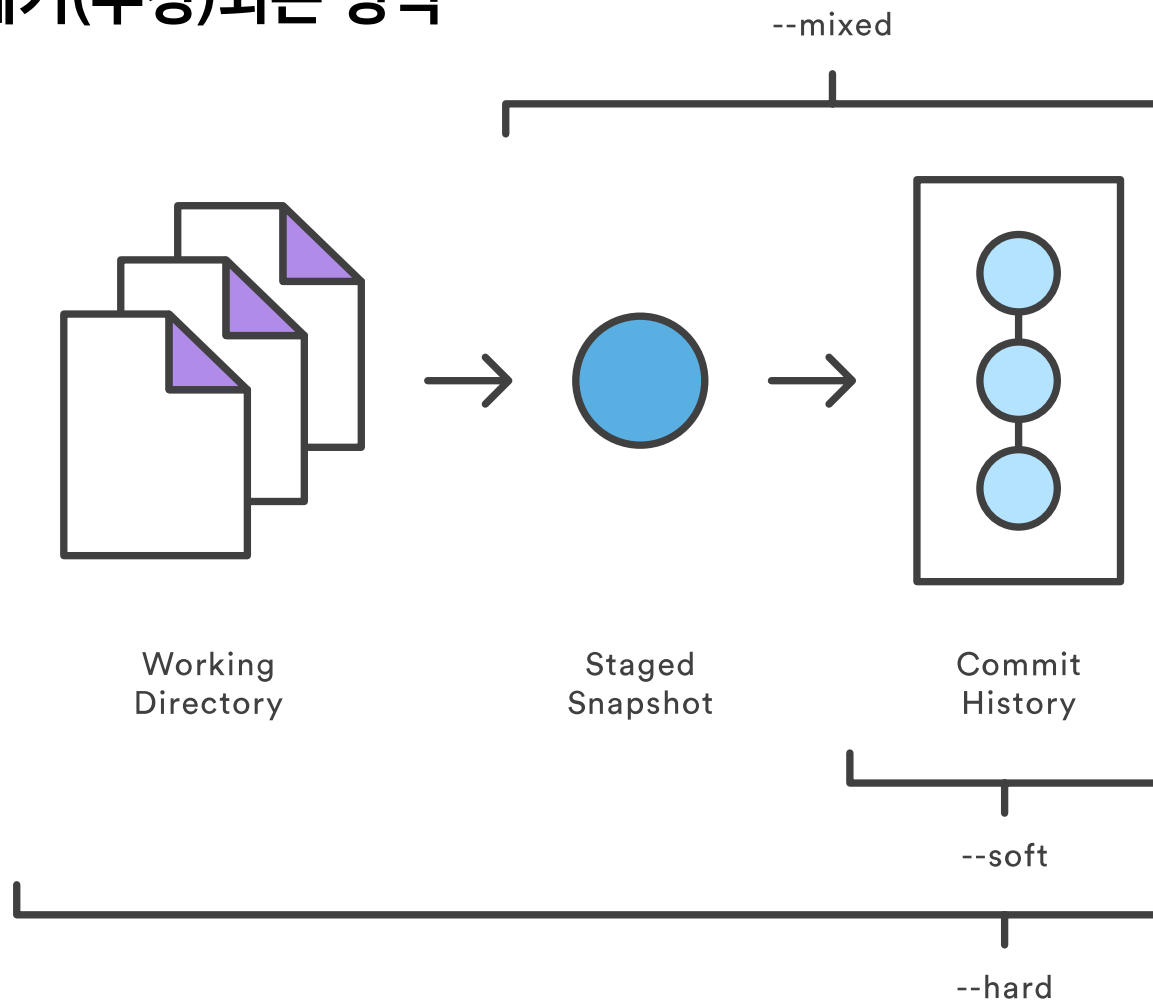
Reset의 3가지 옵션

- --soft, --mixed, --hard
- Reset은 과거 commit으로 되돌아간 후 해당 commit 이후 만들어진 commit들이 삭제됨
- 그런데 삭제되는 commit들의 기록을 어떤 영역에 남겨둘 것인지 옵션을 활용해 조정할 수 있음

옵션 별 동작 방식

- `--soft`
 - 삭제된 commit들의 기록을 staging area에 남김
- `--mixed`
 - 삭제된 commit들의 기록을 working directory에 남김 (기본값)
- `--hard`
 - 삭제된 commit들의 기록을 남기지 않음

Reset의 옵션에 따라 제거(수정)되는 영역



사전 준비

- 제공된 revert-reset-practice.zip에서 reset의 각 폴더에서 진행
- soft, mixed, hard 폴더 모두 같은 git 저장소이며 commit 기록이 같음
- Commit 별 내역

Commit 이름	Commit 내역
first	1.txt가 생성된 사실 기록
second	2.txt가 생성된 사실 기록
third	3.txt가 생성된 사실 기록

- untracked.txt
 - Commit에 포함된 적 없는 파일

soft 옵션 (1/4)

- Commit 목록 확인

```
$ git log --oneline  
d7c8501 (HEAD -> master) third  
91cbd74 second  
f7b3a3d first
```


soft 옵션 (2/4)

- 'first' commit으로 되돌아가기

```
$ git reset --soft f7b3
```

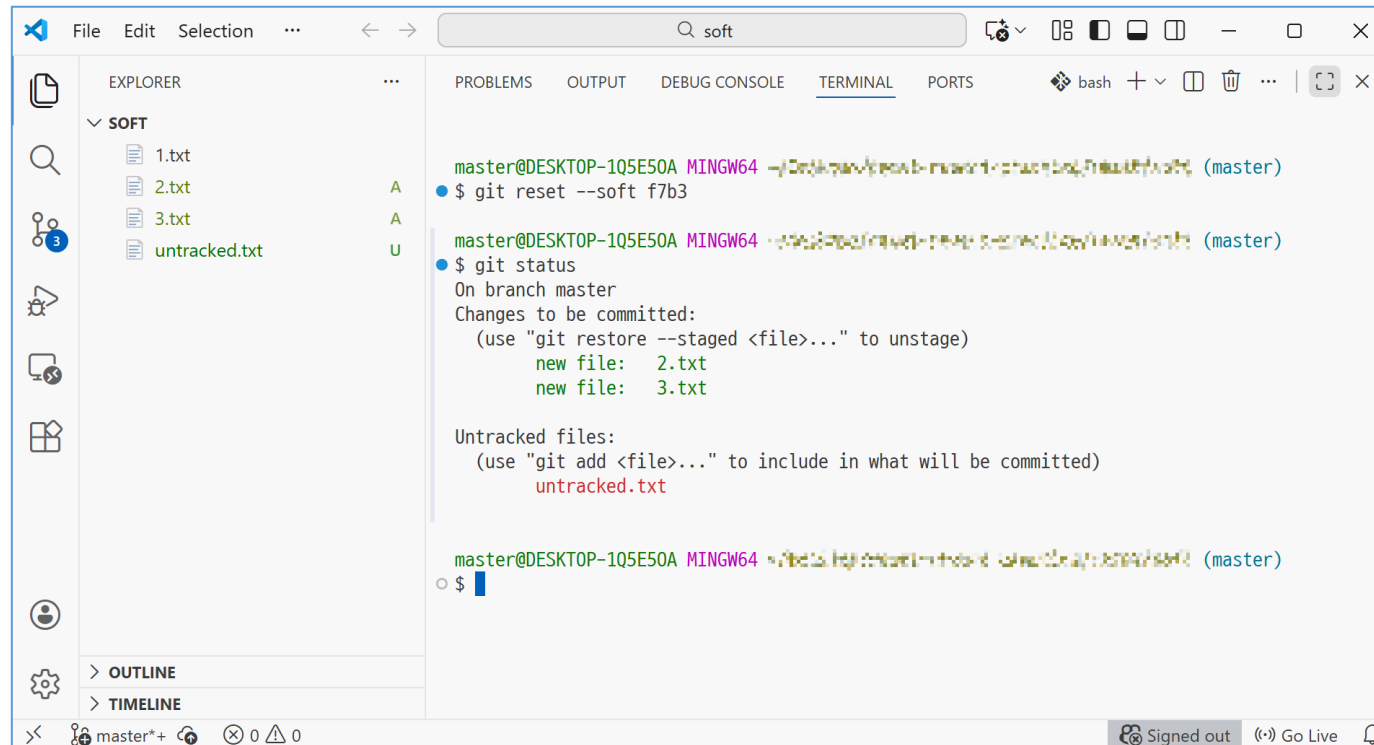
soft 옵션 (3/4)

- reset 후 commit 목록 확인
- 'first' commit 이후에 기록된 'second'와 'third' commit이 삭제

```
$ git log --oneline  
f7b3a3d (HEAD -> master) first
```

soft 옵션 (4/4)

- 'second'와 'third' commit의 기록이 staging area에 남겨짐



The screenshot shows the Visual Studio Code interface. The Explorer view on the left shows a folder named 'SOFT' containing files '1.txt', '2.txt', '3.txt', and 'untracked.txt'. The '2.txt' and '3.txt' files are highlighted in green. The Terminal view on the right shows the following commands and output:

```
master@DESKTOP-1Q5E50A MINGW64 ~\Desktop\people-master\source\code\build\soft (master)
$ git reset --soft f7b3

master@DESKTOP-1Q5E50A MINGW64 ~\Desktop\people-master\source\code\build\soft (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   2.txt
        new file:   3.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        untracked.txt

master@DESKTOP-1Q5E50A MINGW64 ~\Desktop\people-master\source\code\build\soft (master)
$
```

mixed 옵션 (1/4)

- Commit 목록 확인

```
$ git log --oneline  
d7c8501 (HEAD -> master) third  
91cbd74 second  
f7b3a3d first
```

mixed 옵션 (2/4)

- 'first' commit으로 되돌아가기

```
$ git reset --mixed f7b3
```

- ※ mixed가 기본값이므로 '--mixed' 를 생략해도 동일하게 동작함

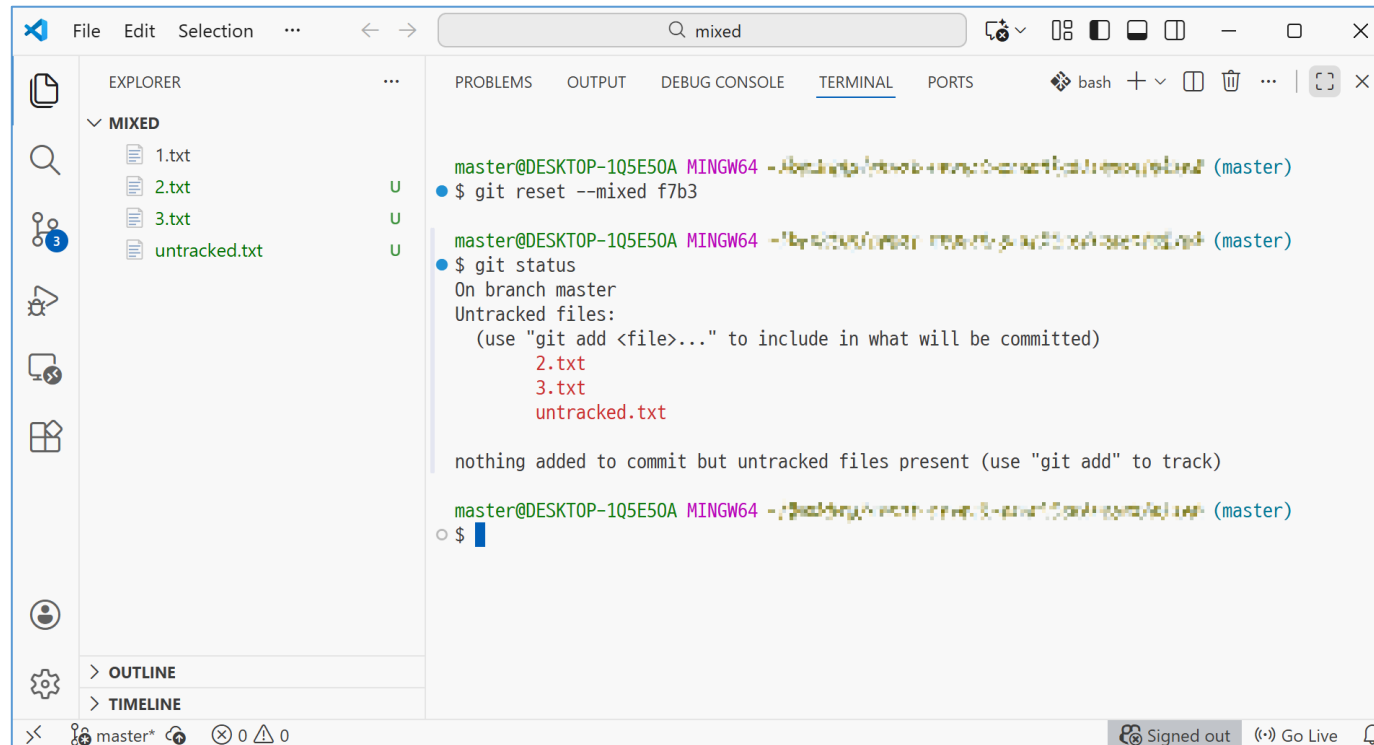
mixed 옵션 (3/4)

- reset 후 commit 목록 확인
- 'first' commit 이후에 기록된 'second'와 'third' commit이 삭제

```
$ git log --oneline  
f7b3a3d (HEAD -> master) first
```

mixed 옵션 (4/4)

- 'second'와 'third' commit의 기록이 working directory에 남겨짐



```
master@DESKTOP-1Q5E50A MINGW64 ~/... (master)
$ git reset --mixed f7b3

master@DESKTOP-1Q5E50A MINGW64 ~/... (master)
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        2.txt
        3.txt
        untracked.txt

nothing added to commit but untracked files present (use "git add" to track)

master@DESKTOP-1Q5E50A MINGW64 ~/... (master)
$
```

hard 옵션 (1/4)

- Commit 목록 확인

```
$ git log --oneline  
d7c8501 (HEAD -> master) third  
91cbd74 second  
f7b3a3d first
```


hard 옵션 (2/4)

- 'first' commit으로 되돌아가기

```
$ git reset --hard f7b3
```

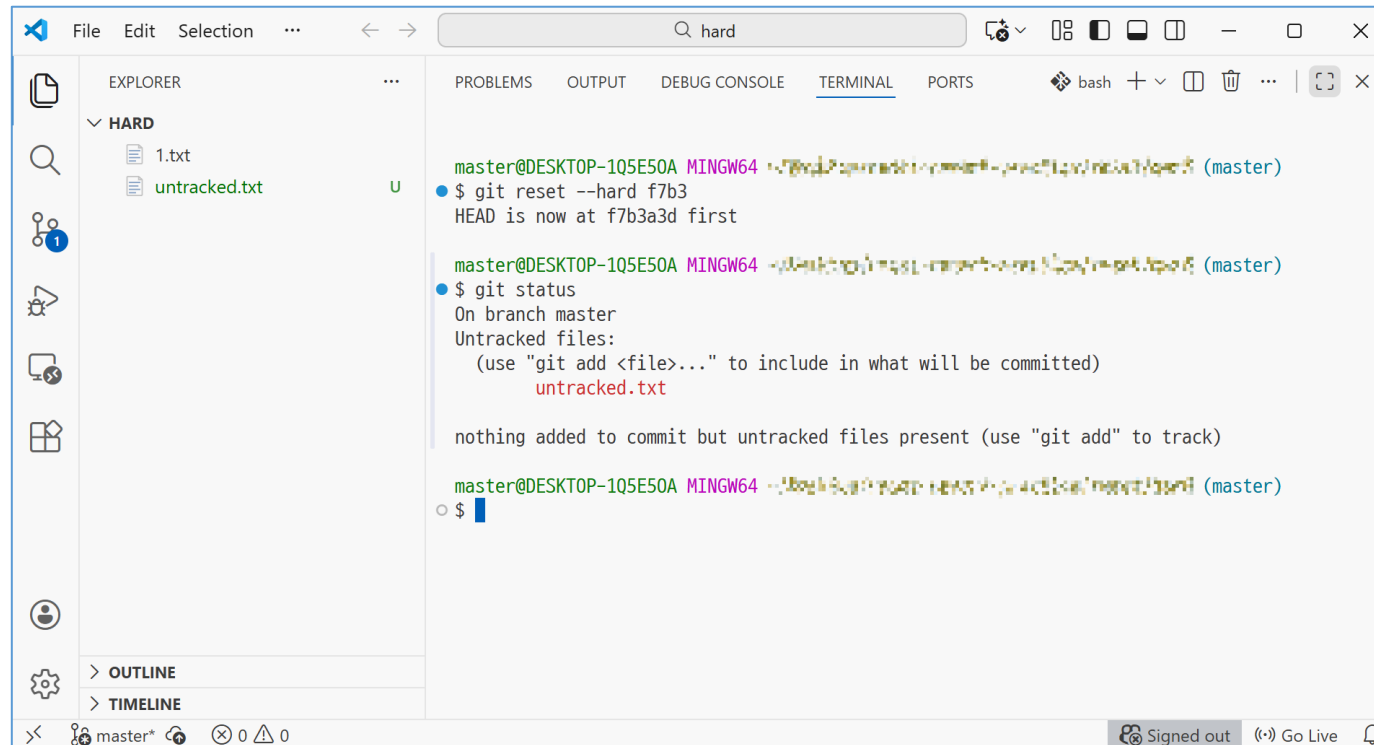
mixed 옵션 (3/4)

- reset 후 commit 목록 확인
- 'first' commit 이후에 기록된 'second'와 'third' commit이 삭제

```
$ git log --oneline  
f7b3a3d (HEAD -> master) first
```

hard 옵션 (4/4)

- 'second'와 'third' commit의 기록된 내역(파일들)이 모두 삭제



```
master@DESKTOP-1Q5E50A MINGW64 ~\Desktop\git-reset\code\hard (master)
$ git reset --hard f7b3
HEAD is now at f7b3a3d first

master@DESKTOP-1Q5E50A MINGW64 ~\Desktop\git-reset\code\hard (master)
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    untracked.txt

nothing added to commit but untracked files present (use "git add" to track)

master@DESKTOP-1Q5E50A MINGW64 ~\Desktop\git-reset\code\hard (master)
$
```

Reset 주의사항

- Reset은 만들어진 commit을 되돌리는 행위
 - Remote에 올라간 commit을 reset 하게 되면?
 - Remote에 기록된 commit과 다른 commit 이력으로 작업하게 됨
 - 이미 Remote의 commit을 가지고 간 동료의 코드가 충돌을 일으킬 수 있음
- Remote에 올라간 commit은 절대 reset 해서는 안됨
- 이미 올라간 commit의 경우 revert를 활용

참고

바로 직전 생성한 commit 수정하기

바로 직전 생성한 commit 수정하기

git commit --amend

1. Commit 메시지 수정
2. Commit 전체 수정

Commit 메시지 수정 - 사전 준비

1. git-amend-practice 폴더 생성
2. 생성한 폴더로 이동
3. VSCode 실행
4. Git 저장소 생성

```
$ mkdir git-amend-practice
```

```
$ cd git-amend-practice
```

```
$ code .
```

```
$ git init
```


Commit 메시지 수정 실습 (1/5)

- A 기능 구현 완료라는 이름의 commit 생성

```
# README.md 파일 생성 후 commit  
$ touch README.md  
  
$ git add .  
$ git commit -m 'A 기능 구현 완료'
```

Commit 메시지 수정 실습 (2/5)

- Commit 목록에서 commit의 hash 값 확인 (2/5)

```
● $ git log --oneline  
7bcb03b (HEAD -> master) A 기능 구현 완료
```

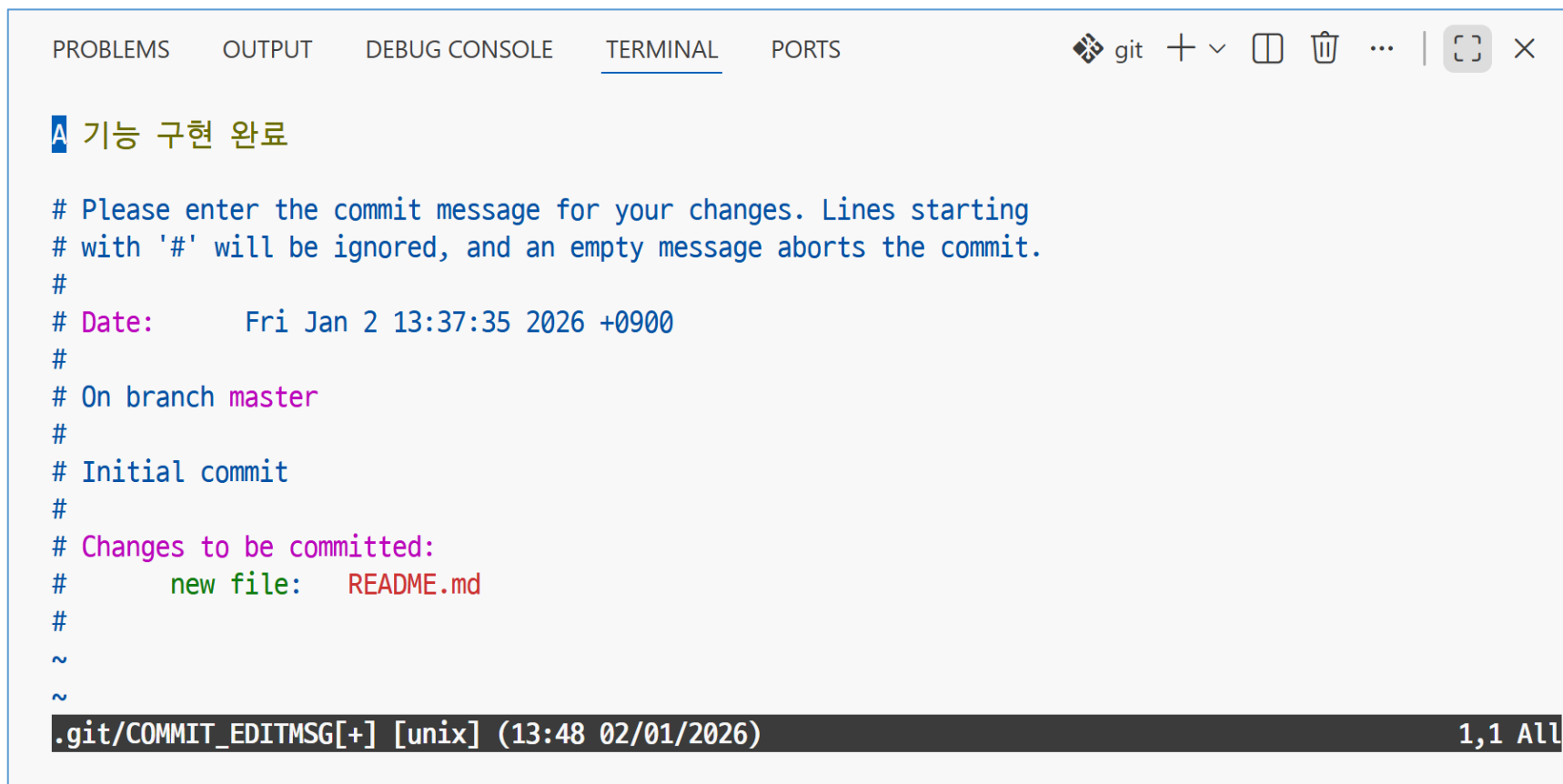
Commit 메시지 수정 실습 (3/5)

- Commit 메시지 수정 진행

```
$ git commit --amend
```

Commit 메시지 수정 실습 (4/5)

- Vim 에디터가 열리면서 직전 commit 메시지를 수정할 수 있음



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
git + - [ ] [X] ... | [ ] X

A 기능 구현 완료

# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# Date:      Fri Jan 2 13:37:35 2026 +0900
#
# On branch master
#
# Initial commit
#
# Changes to be committed:
#   new file:   README.md
#
~
~

.git/COMMIT_EDITMSG[+] [unix] (13:48 02/01/2026) 1,1 All
```

Commit 메시지 수정 실습 (5/5)

- Vim 저장 후 종료
- Commit 메시지 수정 후 변경된 commit 메시지와 commit 해시 값 확인

```
● $ git log --oneline  
7bcb03b (HEAD -> master) A 기능 구현 완료
```

변경 전

```
● $ git log --oneline  
82779fb (HEAD -> master) B 기능 구현 완료
```

변경 후

Commit 전체 수정 - 사전 준비

- 실수로 b-function.txt를 빼고 commit을 생성한 상황이라고 가정
 - b-function.txt 파일 생성

```
$ touch b-function.txt
```

Commit 전체 수정 실습 (1/4)

- b-function.txt 파일을 staging area에 추가

```
$ git add .
```

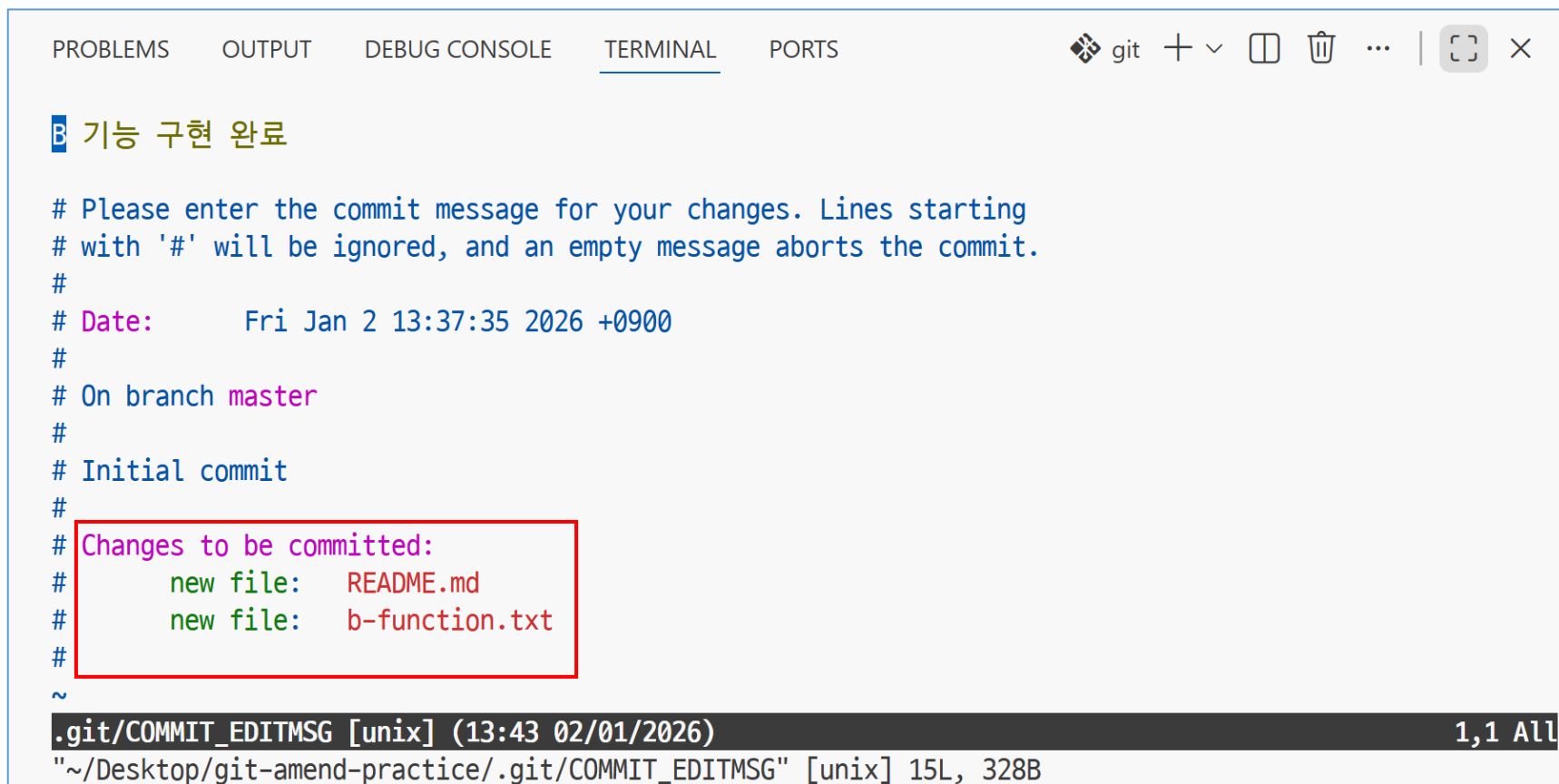
Commit 전체 수정 실습 (2/4)

- Commit 수정 진행

```
$ git commit --amend
```


Commit 전체 수정 실습 (3/4)

- Vim 에디터가 열리면서 새로운 commit 정보를 확인 및 수정할 수 있음



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
B 기능 구현 완료

# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# Date:      Fri Jan 2 13:37:35 2026 +0900
#
# On branch master
#
# Initial commit
#
# Changes to be committed:
#   new file:   README.md
#   new file:   b-function.txt
#
~
.git/COMMIT_EDITMSG [unix] (13:43 02/01/2026) 1,1 All
"~/Desktop/git-amend-practice/.git/COMMIT_EDITMSG" [unix] 15L, 328B
```

Commit 전체 수정 실습 (4/4)

- Vim 저장 후 종료
- Commit 메시지 수정 후 변경된 Commit 메시지와 Commit 해시 값 확인

```
• $ git log --oneline  
82779fb (HEAD -> master) B 기능 구현 완료
```

변경 전

```
• $ git log --oneline  
4d707e2 (HEAD -> master) B 기능 구현 완료 / b-function 파일 추가
```

변경 후

git commit --amend 정리

- 버전 관리 측면에서 봤을 때
"앗, 빠진 파일 넣었음", "이전 commit에서 오타 살짝 고침"과 같은 commit은
유효한 버전이라고 보기 어려움
- 즉, 불필요한 commit을 생성하지 않고, 직전 commit을 수정할 수 있기 때문에
Git에서 꼭 필요한 기능 중에 하나라고 볼 수 있음

파일 내용을 수정 전으로 되돌리기

 git restore

git restore

Modified 상태의 파일 되돌리기

Working Directory에서 파일을 수정했을 때
파일의 수정 사항을 취소하고, 원래 모습대로 되돌리는 작업입니다.

사전 준비 (1/2)

1. git-restore-practice 폴더 생성
2. 생성한 폴더로 이동
3. VSCode 실행
4. Git 저장소 생성

```
$ mkdir git-restore-practice  
  
$ cd git-restore-practice  
  
$ code .  
  
$ git init
```

사전 준비 (2/2)

5. README.md 파일이 git에 의해
버전 관리가 될 수 있도록
첫 commit 작성

```
# README.md 파일 생성
$ touch README.md

# README.md 내용에 '# 제목입니다.' 작성
$ git add .
$ git commit -m "first commit"
```

Restore 실습 (1/2)

1. README.md 파일 추가 수정 후 저장
2. modified 상태 확인

README.md > abc # 제목입니다.

1 # 제목입니다.

2

3 안녕하세요

4

```
● $ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
```


Restore 실습 (2/2)

1. git restore 명령으로 수정 전으로 되돌리기
2. README.md 파일의 내용을 확인

```
git restore README.md
```

❏ README.md > abc # 제목입니다.

1 # 제목입니다.

2

삭제된 내용

Restore 주의사항

- 원래 파일로 덮어쓰는 원리이기 때문에 수정한 내용은 전부 사라짐
- git restore를 통해 수정 취소 후에는 해당 내용을 복원할 수 없음

```
● $ git status  
On branch master  
nothing to commit, working tree clean
```

Staging area에 올라간 파일을 unstage 하기

Unstage 하는 명령어

1. `git restore -staged`
2. `git rm --cached`

git restore --staged

git restore --staged

Staging Area에서 Working Directory로 되돌리기

Staging area의 변경 사항을 원래대로 되돌립니다.

Staging area가 원래대로 되돌아오면서, 그전에 add 했던 내역이 사라지게 됩니다.

restore --staged 사전준비

1. git-restore-staged-practice 폴더 생성
2. 생성한 폴더로 이동
3. 실습을 위해 VSCode 실행
4. Git 저장소 생성
5. README.md 생성 후 commit

```
$ mkdir git-restore-staged-practice  
$ cd git-restore-staged-practice  
$ code .  
  
$ git init  
  
$ touch README.md  
$ git add .  
$ git commit -m "first commit"
```

restore --staged 실습 (1/2)

1. README.md 파일을 다시 수정 후 저장
2. README.md 파일을 staging area에 추가

```
● $ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
       modified:   README.md
```

restore --staged 실습 (2/2)

- git restore --staged README.md 실행 후 결과 확인

```
master@DESKTOP-1Q5E50A MINGW64 ~\Desktop\git\ch02\ch02-02\ch02-02-01 (master)
● $ git restore --staged README.md

master@DESKTOP-1Q5E50A MINGW64 ~\Desktop\git\ch02\ch02-02\ch02-02-01 (master)
● $ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
```


restore --staged 가 안되는 경우

- git restore --staged 명령은 staging area의 변경 사항을 마지막 commit을 기준으로 되돌리는 것
- 만약 되돌아갈 commit이 없다면?

 git rm --cached

git rm --cached

Git이 파악하고 있는 파일을 Git에서만 제거

Staging area 또는 repository에 포함된 파일을 Git에서만 제거합니다.
Staging area에서 제거하고 파일은 유지되기 때문에, 결과적으로 add 가 취소됩니다.

TIP

- git rm 명령의 경우 Git에 저장된 파일을 삭제하는 것이 본래의 목적입니다.
- --cached를 생략하는 경우 해당 파일이 Git에서도 제거되고 파일도 삭제되며, 해당 내역이 add 됩니다.
 - 이미 저장소에 commit 된 파일을 --cached로 제거할 경우, 파일은 남지만 untracked 상태가 됩니다.

rm --cached 사전준비

1. git-rm-cached-practice 폴더 생성
2. 생성한 폴더로 이동
3. 실습을 위해 VSCode 실행
4. Git 저장소 생성
5. README.md 생성 후 add

```
$ mkdir git-rm-cached-practice  
$ cd git-rm-cached-practice  
$ code .  
  
$ git init  
  
$ touch README.md  
$ git add .
```

rm --cached 실습 (1/2)

- git status로 상태 확인
 - No commits yet
 - git restore --staged 작동하지 않음

```
master@DESKTOP-1Q5E50A MINGW64 ~\git\test\git-work\test\test\test (master)
● $ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
       new file:   README.md
```

rm --cached 실습 (2/2)

- git rm --cached README.md 실행 후 결과 확인

```
master@DESKTOP-1Q5E50A MINGW64 ~\Desktop\git\test\test\test\test (master)
● $ git rm --cached README.md
rm 'README.md'

master@DESKTOP-1Q5E50A MINGW64 ~\Desktop\git\test\test\test\test (master)
● $ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
      README.md

nothing added to commit but untracked files present (use "git add" to track)
```

결론

- `git restore --staged`
 - staging area에 추가된 변경 사항을 현재 버전으로 되돌리기
 - add 했던 내역이 취소됨, 변경 사항은 유지
 - 마지막 버전이 있어야 함, 즉 commit이 하나라도 있어야 함
- `git rm --cached`
 - Repository 또는 staging area에서 파일을 제거, 실제 파일은 working directory에 유지
 - Staging area에서만 파일을 제거하게 됨으로, add 했던 내역이 취소되고 파일은 유지
 - 버전을 되돌리는 행위가 아니기 때문에, commit이 없더라도 동작
 - 다른 상황에서 사용할 경우 파일이 Git에서 사라질 수 있음

삭제한 commit으로 돌아가기

이미 삭제한 commit으로 다시 돌아가고 싶다면?

- git reflog
 - HEAD가 이전에 가리켰던 모든 commit을 보여줌
 - Reset의 **--hard** 옵션을 통해 지워진 commit도 reflog로 조회하여 복구 가능

```
$ git reflog
1a410ef HEAD@{0}: reset: moving to 1a410ef
91cbd74 HEAD@{1}: commit: modified repo.rb a bit
f7b3a3d HEAD@{2}: commit: added repo.rb

$ git reset --hard <복구하고자 하는 commit id>
```




감사합니다

